Fakultät für Informatik und Mathematik
Dr. Armin Größlinger
Winter semester 2026/2027

Exercises for Lecture: Virtual Machines and Runtime Systems

Exercise Sheet 1 (Dynamic Libraries in C)

Problem 1 (Addresses)

The program `addrs.c` (see Stud.IP) uses the symbols `etext`, `edata` and `end` to show the bounds of the text, data and BSS segments of a program. Write a small library `libshowaddrs.a`/`libshowaddrs.so` that also declares `etext`, etc. and contains a function that (similar to `addrs.c`) prints the addresses of `etext`, etc. Extend `addrs.c` such that it also calls the output function of the library, i.e., the bounds of the segments are printed twice, once from the main program, once from the library.

(a) Link the program `addrs` staticall with `libshowaddrs.a` (or the source file for `libshowaddrs.a` or the related object file).

(b) Link the program `addrs` with `libshowaddrs.so`, i.e., use the options `-L. -lshowaddrs -Wl,-rpath,.` such that `libshowaddrs.so` is found.

(c) Do *not* link the the main program with `libshowaddrs.so` but use the functions `dlopen` and `dlsym` in `addrs.c` to load `libshowaddrs.so` dynamically and to find (and call) the output function. The flag `-Wl,-rpath,.` is necessary for `dlopen` to find the library.

How do the results of running `addrs` in the three cases compare?

(Note: If supported by your installation, adding the flag `-m32` to the `gcc` command line tells the compiler to compile for a 32-bit target (instead of a 64-bit target); the behaviour may be different between 32-bit and 64-bit targets.)

Problem 2 (Primitive JIT Compiler)

Using the functions `dlopen`, `dlsym`, etc. and a compiler binary it is possible to build a simple just in time compiler. To this end, one writes the code to be compiled into a file (e.g., in C syntax), calls the appropriate compiler (e.g., `gcc`) to obtain a dynamic library and loads this libray with `dlopen` to execute the compiled code.

Write a program (based on the skeleton `eval.c` from Stud.IP) that allows the user to enter an expression (of type `double` in C syntax), evaluates the expression dynamically and prints its value.

Write the user's input together with necessary declarations such as

```
#include <math.h>
double f(void) {  return  <user input> ;  }
```

to a file, call `gcc` to compile the file and then call `f` in the generated library to obtain the value of the expression.

You can use the C functions `fopen`, `fprintf`, `fclose`, `system` to manipulate a file and call the compiler (see also the corresponding man pages of these functions, e.g., `man fopen`).

You can hand your programs in by uploading them in the folder "Submissions" in Stud.IP (hand-in is optional).

Due date: Thursday, October 22 at 18:00, discussion on October 23, 2026